SIMATS ENGINEERING

CO5- ASSESSMENT TOOL 2 – Technical Presentation

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 10%

Time Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I V.Sireesha(192472208) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
V.Sireesha

1. System Analysis Presentation

Present a reverse engineering analysis of an AI-based system, explaining its architecture, components, and working process using diagrams.

2. Architecture Reconstruction Demo

Prepare a technical presentation to reconstruct the architecture of a given software/AI system, highlighting module interactions and data flow.

3. Algorithm & Logic Identification

Present how you identified algorithms and decision-making logic in a reverse-engineered system, including methodology and tools used.

4. Data Flow & Processing Pipeline

Explain through a presentation the data flow, processing pipeline, and communication mechanisms in a reverse-engineered distributed system.

5. Enhancement & Redesign Proposal

Deliver a technical presentation proposing improvements or redesign strategies for an existing system based on reverse engineering findings.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

1. System Analysis Presentation

Title: Reverse Engineering Analysis of an AI-Based System

Introduction

- Reverse engineering is the process of analyzing an existing AI system to understand its architecture, components, algorithms, data flow, and working process.
- The selected system is analyzed without modifying its original functionality.
- The main objective is to understand how different components communicate and how input data is transformed into the final output.

Problem Identification

- The system may have complex or undocumented internal components.
- Understanding the relationship between input, processing modules, AI algorithms, and output is difficult.
- Reverse engineering helps identify system dependencies, processing bottlenecks, and possible improvement areas.

System Architecture

User / Input

↓

Data Collection Layer

↓

Data Preprocessing

↓

AI / ML Processing

↓

Decision-Making Module

↓

Output / Response

Major Components

1. **Input Layer** – Collects user or system data.
2. **Data Processing Layer** – Cleans and transforms the input.
3. **AI Processing Layer** – Applies AI/ML algorithms.
4. **Decision Layer** – Produces decisions or predictions.
5. **Output Layer** – Displays or communicates the result.
6. **Storage Layer** – Stores required data, models, and results.

Working Process

1. Input is received from the user or external source.
2. Data is validated and preprocessed.
3. Processed data is supplied to the AI model.
4. The model performs prediction or decision-making.
5. The decision module interprets the result.
6. The final output is returned to the user.

Tools Used

- Python
- PySpark
- RDD operations
- System monitoring tools
- Flow/architecture diagrams
- Log analysis

Conclusion

- Reverse engineering provides a clear understanding of an AI system's architecture and working process.
- It helps identify dependencies, performance issues, and opportunities for redesign.

2. Architecture Reconstruction Demo

Title: Reconstruction of AI System Architecture

Objective

- To reconstruct the architecture of an existing AI system.
- To identify individual modules and understand their interactions.
- To represent the complete data flow from input to output.

Problem Identification

- Existing systems may contain undocumented modules and dependencies.
- Directly understanding the complete architecture can be difficult.
- Architecture reconstruction provides a structured representation of the system.

Reconstructed Architecture

User/Input

↓

Data Collection

↓

Preprocessing

↓

AI Processing

↓

Decision Module

↓

Output / Result

Module Interactions

- The input module sends raw data to preprocessing.
- Preprocessing converts raw data into usable information.
- The AI module processes the prepared data.
- The decision module interprets the AI result.

- The output module provides the final response.
- Storage components can support multiple modules by providing historical data.

Data Flow

Raw Data

↓

Cleaning

↓

Transformation

↓

Feature/Data Representation

↓

AI Processing

↓

Decision

↓

Final Output

Tools

- Architecture diagrams
- Python
- PySpark
- Log analysis
- Performance monitoring
- Data-flow visualization

Conclusion

- Architecture reconstruction makes hidden relationships and dependencies easier to understand.
- It provides a foundation for testing, optimization, and redesign.

3. Algorithm & Logic Identification

Title: Identification of Algorithms and Decision-Making Logic

Objective

- To identify the algorithms used inside a reverse-engineered AI system.
- To understand how the system makes decisions.
- To determine the relationship between input data, processing logic, and output.

Methodology

1. **Input Analysis**
    - Identify the type and format of input data.
    - Determine how the system receives the data.
2. **Data-Flow Analysis**
    - Trace the movement of data through different modules.
    - Identify transformations performed on the data.
3. **Code/Logic Analysis**
    - Examine available source code, functions, modules, and dependencies.
    - Identify loops, conditions, mathematical operations, and decision rules.
4. **Algorithm Identification**
    - Compare observed processing patterns with known algorithms.
    - Identify AI/ML algorithms based on their operations and outputs.

5. **Output Analysis**
    - Compare inputs with generated outputs.
    - Determine how processing decisions affect the final result.

Decision-Making Flow

Input Data

↓

Preprocessing

↓

Feature/Data Extraction

↓

Algorithm

↓

Prediction / Score

↓

Decision Rules

↓

Final Output

Tools Used

- Python
- PySpark
- RDD operations
- Debuggers
- Log analysis
- Code analysis
- Performance monitoring

Validation

- Test the system with different inputs.
- Compare expected and actual outputs.
- Analyze processing time and resource usage.
- Verify whether identified algorithms consistently explain system behavior.

Conclusion

- Algorithm identification helps explain the intelligence behind the system.
- Reverse engineering makes the decision-making process more transparent and helps identify areas for optimization.

4. Data Flow & Processing Pipeline

Title: Data Flow and Processing Pipeline in a Distributed AI System

Introduction

- Distributed AI systems process large amounts of data using multiple computing resources.
- Data is divided, processed, transferred, and combined to produce the final result.

Problem Identification

- Large datasets can make centralized processing slow.
- Communication between distributed components can introduce delays.
- Data consistency and fault handling must be maintained.

Processing Pipeline

Data Sources

↓

Data Ingestion

↓

Data Partitioning

↓

Distributed Processing

↓

AI/ML Computation

↓

Aggregation

↓

Result Storage

↓

User/Application

PySpark/RDD-Based Processing

Input Dataset

↓

RDD Creation

↓

Partitioning

↓

Transformation (map, filter, etc.)

↓

Action

(reduce, collect, etc.)

↓

Final Result

Communication Mechanisms

- Data is transferred between distributed processing components.
- The system coordinates tasks across multiple nodes.
- Intermediate results may be exchanged between processing stages.
- The final results are aggregated and delivered to the application.

Important Factors

1. Data partitioning
2. Parallel processing
3. Network communication
4. Fault tolerance
5. Resource utilization
6. Result aggregation

Tools

- PySpark
- RDD
- Spark monitoring tools
- Logs
- Performance monitoring
- Data-flow diagrams

Conclusion

- A distributed processing pipeline improves scalability and processing efficiency.
- Understanding data flow helps identify bottlenecks and optimize communication between components.

5. Enhancement & Redesign Proposal

Title: Enhancement and Redesign of an Existing AI System

Objective

- To identify limitations discovered during reverse engineering.
- To propose technical improvements.
- To redesign the system for better performance, scalability, reliability, and maintainability.

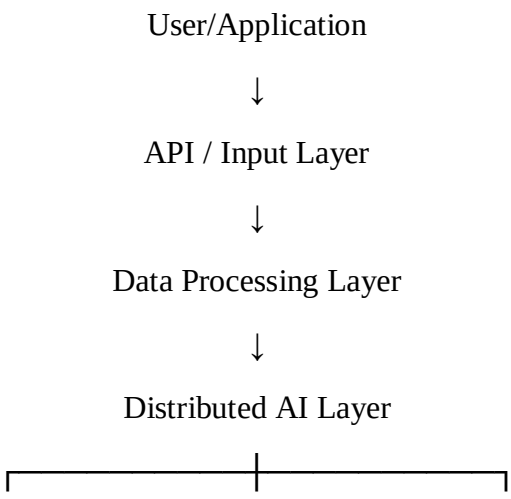
Problems Identified

- High processing time for large datasets.
- Excessive resource utilization.
- Communication overhead between distributed components.
- Difficult maintenance due to tightly connected modules.
- Limited monitoring and error analysis.

Proposed Improvements

Existing Issue	Proposed Enhancement
Slow data processing	Use parallel distributed processing
Large data volume	Use PySpark/RDD-based processing
High communication overhead	Optimize data transfer
Poor scalability	Introduce scalable architecture
Difficult debugging	Improve logging and monitoring
Resource wastage	Optimize CPU and memory usage
Single point of failure	Add fault-tolerant mechanisms

Redesigned Architecture



↓ ↓ ↓

Worker 1 Worker 2 Worker 3

↓

Result Aggregation

↓

Output Layer

↓

Storage/Reports

Expected Benefits

1. Faster processing.
2. Better scalability.
3. Improved resource utilization.
4. Higher system reliability.
5. Easier maintenance.
6. Better monitoring and debugging.
7. Improved handling of large datasets.

Ethical and Professional Considerations

- Protect sensitive data during processing.
- Maintain transparency in AI-based decisions.
- Validate system outputs before deployment.
- Avoid unauthorized modification of existing systems.
- Document all reverse-engineering findings and redesign decisions.

Conclusion

- The redesigned architecture addresses the limitations identified during reverse engineering.
- Using scalable distributed processing and improved monitoring can make the AI system more efficient, reliable, and maintainable.